Nama : Hanifah Hasanah
NIM : 123140082
Github:
https://github.com/HANIFAHHASANAH-123140082/sig_123140082_Hanifah-Hasanah.git
TUGAS 7

A. Deskripsi Tugas

Bangun REST API lengkap untuk mengakses data spasial dari database PostGIS menggunakan FastAPI. API harus mendukung operasi CRUD dan query spasial.

Ketentuan:

- Implementasi minimal 5 endpoint (GET all, GET by id, GET geojson, POST, nearby)
- Gunakan Pydantic untuk validasi input
- Output format GeoJSON untuk endpoint /geojson
- Implementasi query spasial (nearby atau bbox)
- Dokumentasikan dengan screenshot Swagger UI

B. Langkah Pengerjaan

1. Setup Database PostGIS

Langkah pertama adalah menyiapkan database yang akan menyimpan data spasial fasilitas. PostGIS adalah ekstensi PostgreSQL yang memungkinkan penyimpanan dan query data geografis seperti koordinat titik (Point).

Query | Query History

```sql
CREATE EXTENSION IF NOT EXISTS postgis;

CREATE TABLE fasilitas (
    id SERIAL PRIMARY KEY,
    nama VARCHAR(255) NOT NULL,
    jenis VARCHAR(100) NOT NULL,
    alamat TEXT,
    geom GEOMETRY(Point, 4326)
);
```

Data Output | Messages | Notifications

Showing rows: 1 to 5

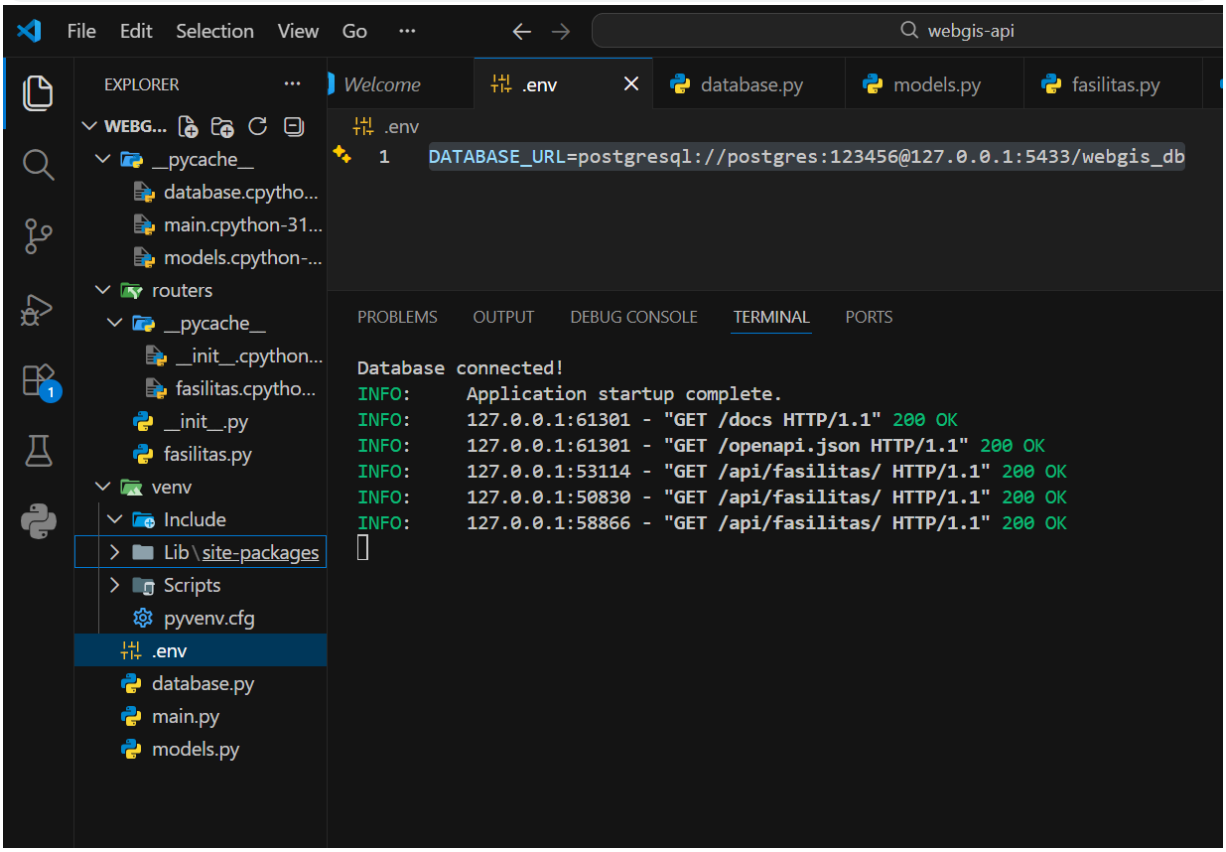
	id [PK] integer	nama character varying (255)	jenis character varying (100)
1	1	RS Abdul Moeloek	Rumah Sakit
2	2	SMAN 1 Bandar Lampung	Sekolah
3	3	Apotek Kimia Farma	Apotek
4	4	Puskesmas Rajabasa	Puskesmas
5	5	Pasar Bambu Kuning	Pasar

2. Setup Project FastAPI

Project FastAPI distruktur menjadi beberapa file dengan fungsi masing-masing agar kode lebih rapi dan mudah dikembangkan. Virtual environment digunakan agar dependency project tidak bentrok dengan package Python lain di komputer.

Perintah instalasi dependencies:

```
None
pip install fastapi uvicorn asyncpg python-dotenv
```



3. Konfigurasi Koneksi Database (database.py)

File database.py bertugas mengelola koneksi ke database PostgreSQL menggunakan connection pool. Connection pool berarti aplikasi tidak membuat koneksi baru setiap ada request, melainkan menggunakan kembali koneksi yang sudah ada sehingga lebih efisien dan cepat.

asyncpg dipilih karena mendukung operasi async/await yang cocok dengan FastAPI, berbeda dengan psycopg2 yang bersifat synchronous.

```python
import asyncpg
from dotenv import load_dotenv
import os

load_dotenv()

DATABASE_URL = os.getenv("DATABASE_URL")

pool = None

async def get_pool():
    global pool
    if pool is None:
        pool = await asyncpg.create_pool(
            DATABASE_URL,
            min_size=5,
            max_size=20
        )
    return pool

async def close_pool():
    global pool
    if pool:
        await pool.close()
        pool = None
```

4. Validasi Data dengan Pydantic (models.py)

Pydantic digunakan untuk memvalidasi data yang dikirim oleh pengguna ke API. Jika data tidak sesuai (misalnya longitude melebihi 180 atau nama kurang dari 3 karakter), API akan otomatis menolak dan mengirim pesan error yang jelas tanpa perlu menulis kode validasi manual.

```python
from pydantic import BaseModel, Field
from typing import Optional

class FasilitasCreate(BaseModel):
    nama: str = Field(..., min_length=3)
```

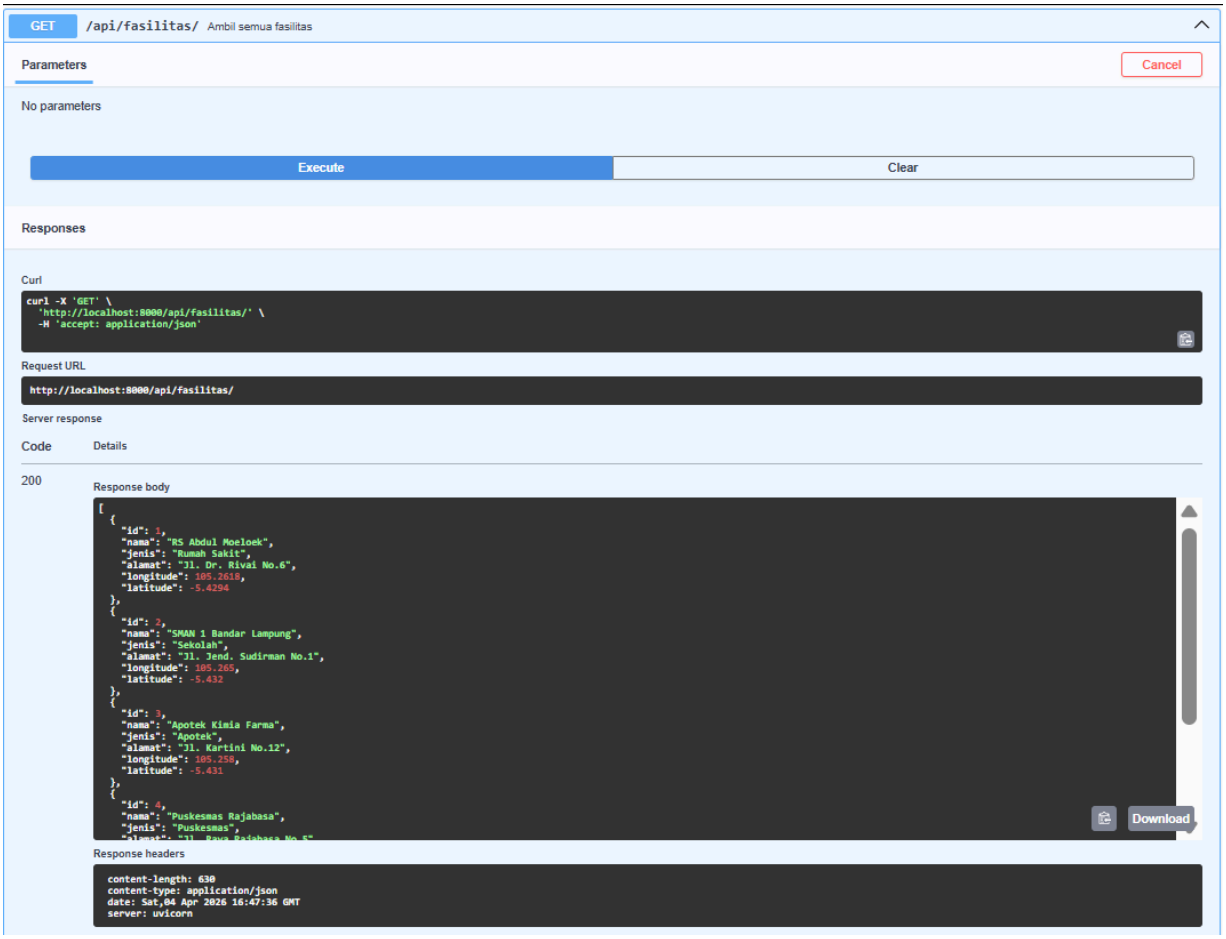
```python
    jenis: str
    alamat: Optional[str] = None
    longitude: float = Field(..., ge=-180, le=180)
    latitude: float = Field(..., ge=-90, le=90)
```

5. Implementasi Endpoint API (routers/fasilitas.py):

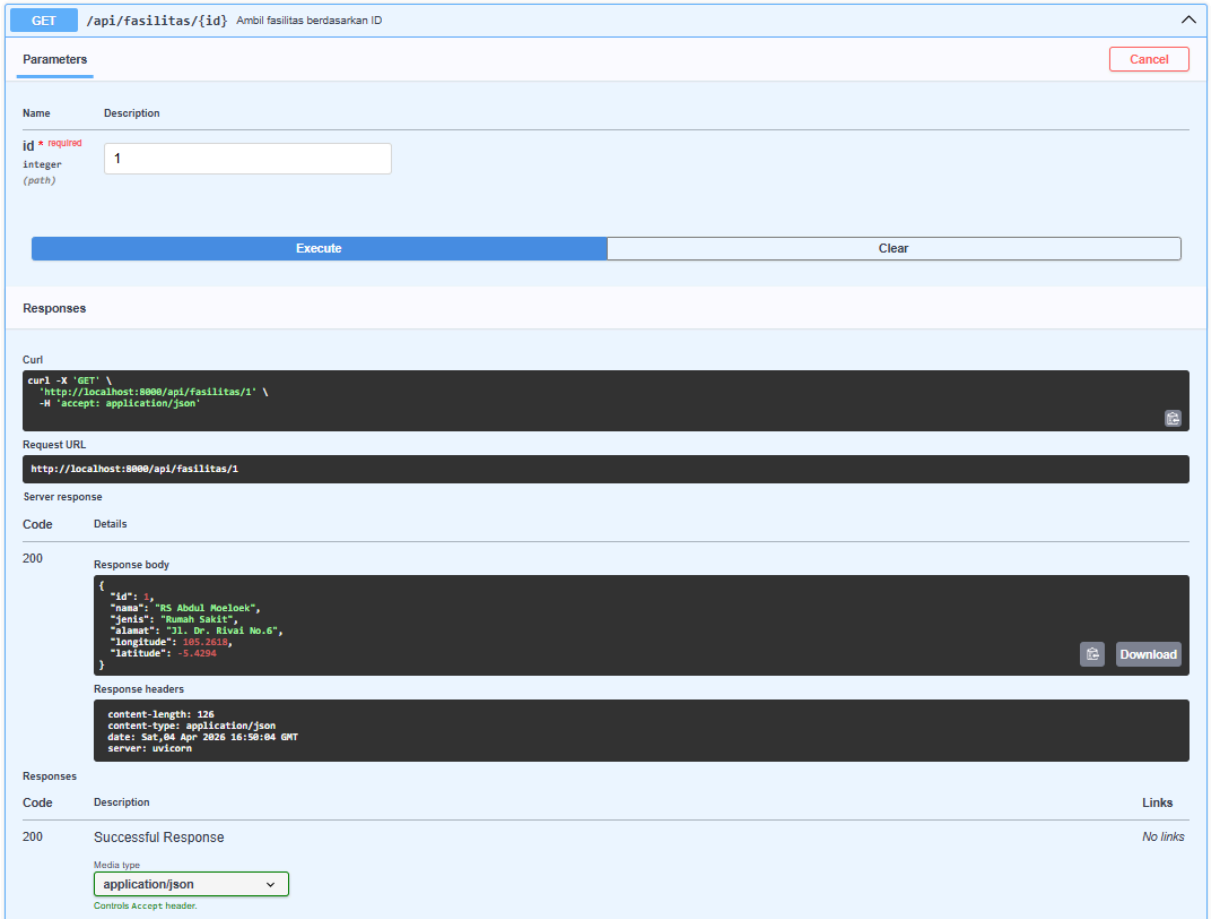
Berikut penjelasan tiap endpoint yang diimplementasikan:

Endpoint 1 - GET /api/fasilitas/

Fungsi: Mengambil seluruh data fasilitas dari database dan mengembalikannya dalam format JSON. Menggunakan ST_X(geom) dan ST_Y(geom) untuk mengekstrak nilai longitude dan latitude dari kolom geometri PostGIS.



Endpoint 2 - GET /api/fasilitas/{id}
Fungsi: Mengambil satu data fasilitas berdasarkan ID. Jika ID tidak ditemukan, API mengembalikan error 404 Not Found. Menggunakan parameterized query ($1) untuk mencegah SQL injection.



Endpoint 3 - GET /api/fasilitas/geojson/all
Fungsi: Mengambil semua data fasilitas dalam format GeoJSON FeatureCollection. Format ini adalah standar internasional untuk data geografis dan dapat langsung digunakan oleh library peta seperti Leaflet dan MapLibre di frontend. Menggunakan fungsi PostGIS ST_AsGeoJSON() untuk mengkonversi geometri ke format GeoJSON.

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:8000/api/fasilitas/geojson/all' \
  -H 'accept: application/json'
```

Request URL

```
http://localhost:8000/api/fasilitas/geojson/all
```

Server response

Code	Details
200	Response body


```
{
  "type": "FeatureCollection",
  "features": [
    {
      "type": "Feature",
      "geometry": {
        "type": "Point",
        "coordinates": [
          105.2618,
          -5.4294
        ]
      },
      "properties": {
        "id": 1,
        "nama": "RS Abdul Moeloek",
        "jenis": "Rumah Sakit",
        "alamat": "Jl. Dr. Rivai No.6"
      }
    },
    {
      "type": "Feature",
      "geometry": {
        "type": "Point",
        "coordinates": [
          105.265,
          -5.432
        ]
      },
      "properties": {
```


Download

Response headers

```
content-length: 935
content-type: application/json
date: Sat,04 Apr 2026 16:51:42 GMT
server: uvicorn
```

Responses

Code	Description	Links
200	Successful Response	*No links*

Media type

application/json

Controls Accept header.

Example Value | Schema

```
"string"
```

Endpoint 4 - GET /api/fasilitas/nearby/search

Fungsi: Mencari fasilitas yang berada dalam radius tertentu dari titik koordinat yang diberikan. Menggunakan fungsi PostGIS ST_DWithin dengan tipe geography (bukan geometry) agar jarak dihitung dalam satuan meter yang akurat di permukaan bumi. Hasil diurutkan dari yang terdekat.

Parameter yang digunakan:

- lat — latitude titik pusat pencarian
- lon — longitude titik pusat pencarian
- radius — jarak maksimum dalam meter (default 2000m)

GET /api/fasilitas/nearby/search Cari fasilitas terdekat

Parameters

Cancel

Name	Description
lat * required number (query)	-5.4294
lon * required number (query)	105.2618
radius integer (query)	3000

Execute | Clear

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:8000/api/fasilitas/nearby/search?lat=-5.4294&lon=105.2618&radius=3000' \
  -H 'accept: application/json'
```

Request URL

```
http://localhost:8000/api/fasilitas/nearby/search?lat=-5.4294&lon=105.2618&radius=3000
```

Server response

Code	Details
200	Response body


```
[
  {
    "id": 1,
    "nama": "RS Abdul Moeloek",
    "jenis": "Rumah Sakit",
    "alamat": "Jl. Dr. Rivai No.6",
    "longitude": 105.2618,
    "latitude": -5.4294,
    "jarak_meter": 0
  },
  {
    "id": 5,
    "nama": "Pasar Bambu Kuning",
    "jenis": "Pasar",
    "alamat": "Jl. Imam Bonjol No.3",
    "longitude": 105.263,
    "latitude": -5.428,
    "jarak_meter": 204
  },
```


200 Response body


```
    "jarak_meter": 0
  },
  {
    "id": 5,
    "nama": "Pasar Bambu Kuning",
    "jenis": "Pasar",
    "alamat": "Jl. Imam Bonjol No.3",
    "longitude": 105.263,
    "latitude": -5.428,
    "jarak_meter": 204
  },
  {
    "id": 2,
    "nama": "SMAN 1 Bandar Lampung",
    "jenis": "Sekolah",
    "alamat": "Jl. Jend. Sudirman No.1",
    "longitude": 105.265,
    "latitude": -5.432,
    "jarak_meter": 457
  },
  {
    "id": 3,
    "nama": "Apotek Kimia Farma",
    "jenis": "Apotek",
    "alamat": "Jl. Kartini No.12",
    "longitude": 105.258,
    "latitude": -5.431,
    "jarak_meter": 457
  }
]
```


Download

Response headers

```
content-length: 573
content-type: application/json
date: Sat,04 Apr 2026 16:53:39 GMT
server: uvicorn
```

Responses

Code	Description	Links
200	Successful Response Media type: application/json Controls Accept header. Example Value | Schema "string"	No links
422	Validation Error Media type: application/json	No links

Endpoint 5 - GET /api/fasilitas/bbox/search

Fungsi: Mencari fasilitas dalam area bounding box (kotak persegi panjang) berdasarkan koordinat minimum dan maksimum. Berguna untuk memuat data sesuai area yang sedang ditampilkan di peta frontend. Menggunakan operator && dan fungsi ST_MakeEnvelope dari PostGIS.

GET /api/fasilitas/bbox/search Cari fasilitas dalam bounding box

Parameters

Cancel

Name	Description
minx * required number (query)	105.2
miny * required number (query)	-5.5
maxx * required number (query)	105.3
maxy * required number (query)	-5.4

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:8000/api/fasilitas/bbox/search?minx=105.2&miny=-5.5&maxx=105.3&maxy=-5.4' \
  -H 'accept: application/json'
```

Request URL

```
http://localhost:8000/api/fasilitas/bbox/search?minx=105.2&miny=-5.5&maxx=105.3&maxy=-5.4
```

Server response

Code Details

200

Response body


```
[
  {
    "id": 1,
    "nama": "RS Abdul Moeloek",
    "jenis": "Rumah Sakit",
    "alamat": "Jl. Dr. Rivai No.6",
    "longitude": 105.2618,
    "latitude": -5.4294
  },
  {
    "id": 2,
    "nama": "SMAN 1 Bandar Lampung",
    "jenis": "Sekolah",
    "alamat": "Jl. Jend. Sudirman No.1",
    "longitude": 105.265,
    "latitude": -5.432
  },
  {
    "id": 3,
    "nama": "Apotek Kimia Farma",
    "jenis": "Apotek",
    "alamat": "Jl. Kartini No.12",
    "longitude": 105.258,
    "latitude": -5.431
  },
  {
    "id": 5,
    "nama": "Pasar Bambu Kuning",
    "jenis": "Pasar",
    "alamat": "Jl. Imam Bonjol No.3",
    "longitude": 105.263,
    "latitude": -5.428
  }
]
```


Download

Response headers

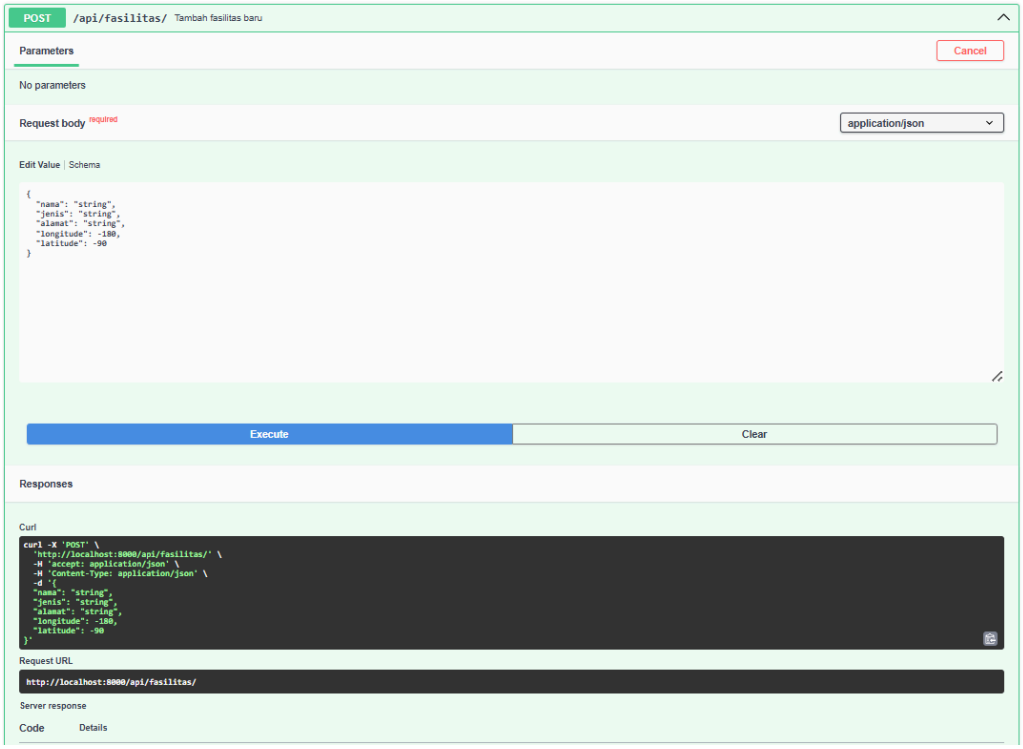
```
content-length: 503
content-type: application/json
date: Sat,04 Apr 2026 16:56:45 GMT
server: uvicorn
```

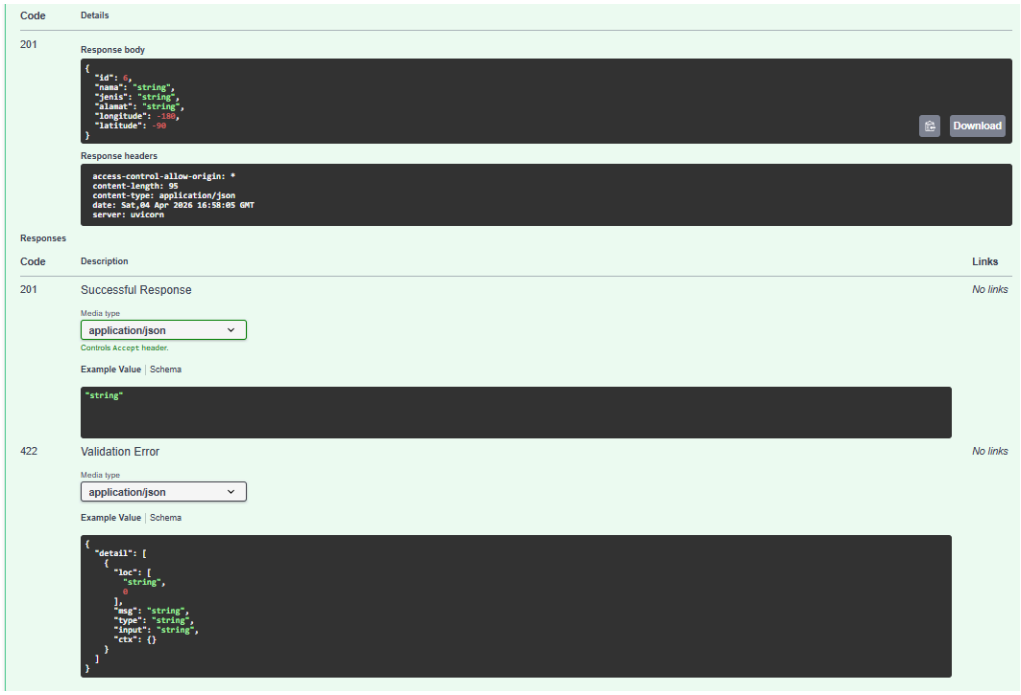
Responses

Code	Description	Links
200	Successful Response	No links

Endpoint 6 - POST /api/fasilitas/

Fungsi: Menambahkan data fasilitas baru ke database. Data dikirim dalam format JSON di request body. Pydantic secara otomatis memvalidasi data sebelum disimpan. Koordinat longitude dan latitude dikonversi ke format geometri PostGIS menggunakan ST_SetSRID(ST_Point(...), 4326). Mengembalikan response 201 Created beserta data yang baru tersimpan termasuk ID auto-generated.





Endpoint 7 - PUT /api/fasilitas/{id}

Fungsi: Mengubah data fasilitas yang sudah ada berdasarkan ID. Jika ID tidak ditemukan, mengembalikan error 404.

PUT /api/fasilitas/{id} Update data fasilitas

Parameters

Cancel

Name	Description
id * required integer (path)	1

Request body required

application/json

Edit Value | Schema

```
{
  "nama": "string",
  "jenis": "string",
  "alamat": "string",
  "longitude": -180,
  "latitude": -90
}
```

Execute Clear

Responses

Curl

```
curl -X 'PUT' \
  'http://localhost:8000/api/fasilitas/1' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
  "nama": "string",
  "jenis": "string",
  "alamat": "string",
  "longitude": -180,
  "latitude": -90
}'
```

Request URL

```
http://localhost:8000/api/fasilitas/1
```

Server response

Code	Details
200	Response body

```
{
  "id": 1,
  "nama": "string",
  "jenis": "string",
  "alamat": "string",
  "longitude": -180,
  "latitude": -90
}
```

Download

Response headers

```
access-control-allow-origin: *
content-length: 95
content-type: application/json
date: Sat,04 Apr 2026 17:00:02 GMT
server: uvicorn
```

Responses

Code	Description	Links
200	Successful Response	No links

Media type

application/json

Controls Accept header.

Example Value | Schema

```
"string"
```

422	Validation Error	No links

Media type

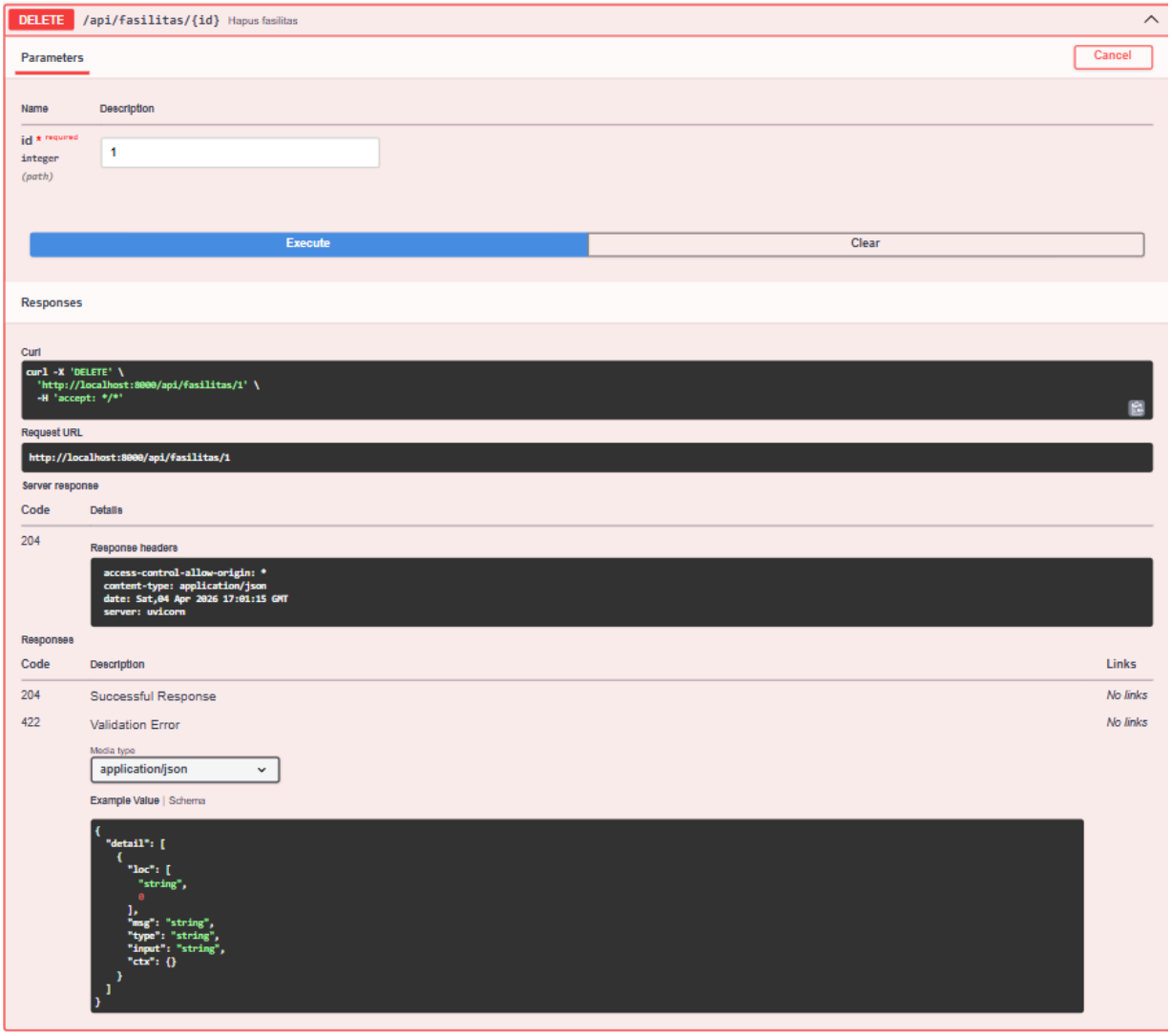
application/json

Example Value | Schema

```
{
  "detail": [
    {
      "loc": [
        "string",
        0
      ],
      "msg": "string",
      "type": "string",
      "input": "string",
      "ctx": {}
    }
  ]
}
```

Endpoint 8 - DELETE /api/fasilitas/{id}

Fungsi: Menghapus data fasilitas berdasarkan ID. Mengembalikan response 204 No Content (tanpa body) sesuai konvensi REST API.



6. Menjalankan Server

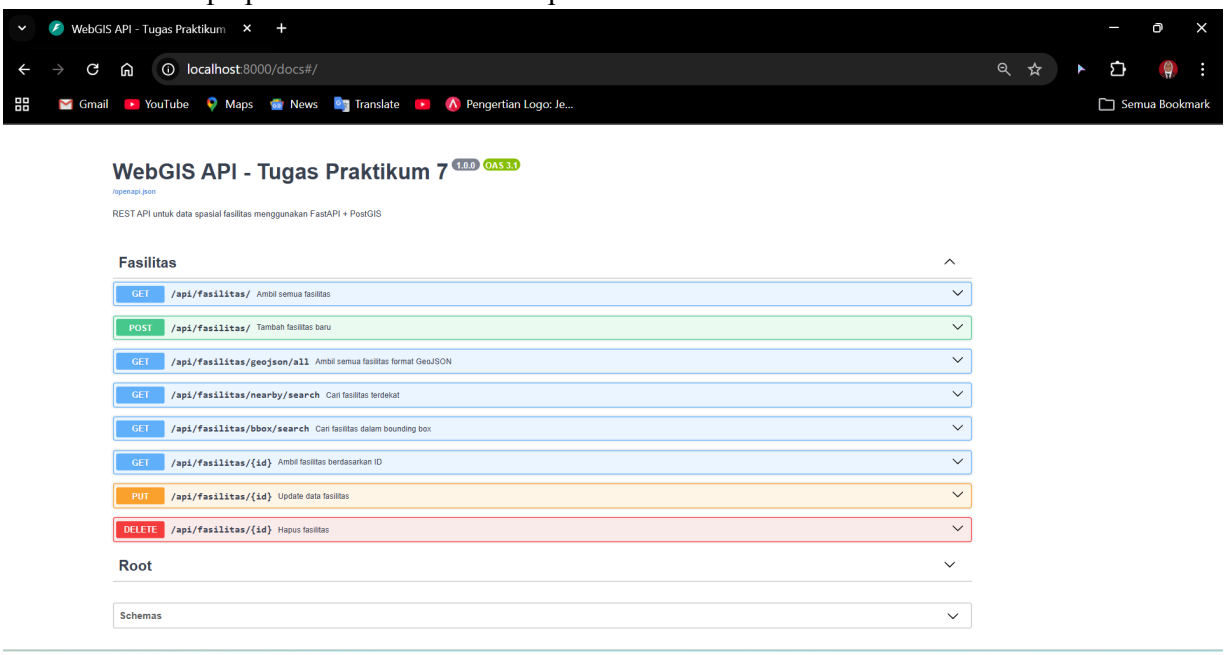
Server dijalankan menggunakan Uvicorn dengan flag --reload yang membuat server otomatis restart setiap ada perubahan kode, sangat berguna saat development.

```
None
uvicorn main:app --reload
```

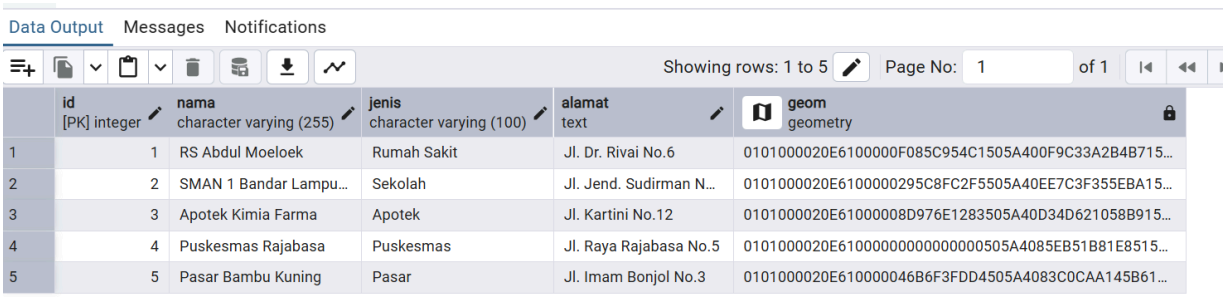
```
(venv) PS C:\Users\sn5wi\Downloads\webgis-api> uvicorn main:app --reload
INFO:     Will watch for changes in these directories: ['C:\\Users\\sn5wi\\Downloads\\webgis-api']
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:     Started reloader process [22432] using StatReload
INFO:     Started server process [15988]
INFO:     Waiting for application startup.
Database connected!
INFO:     Application startup complete.
```

7. Dokumentasi API via Swagger UI

FastAPI secara otomatis menghasilkan dokumentasi interaktif di /docs menggunakan Swagger UI. Fitur ini memungkinkan siapapun untuk melihat dan mencoba semua endpoint langsung dari browser tanpa perlu tools tambahan seperti Postman.



Berikut adalah hasil eksekusi SQL di pgAdmin 4 yang menunjukkan data berhasil tersimpan di database webgis_db



Data Output | Messages | Notifications

Showing rows: 1 to 5 | Page No: 1 of 1

	id [PK] integer	nama character varying (255)	jenis character varying (100)	alamat text	geom geometry
1	1	RS Abdul Moeloek	Rumah Sakit	Jl. Dr. Rivai No.6	0101000020E6100000F085C954C1505A400F9C33A2B4B715...
2	2	SMAN 1 Bandar Lampu...	Sekolah	Jl. Jend. Sudirman N...	0101000020E6100000295C8FC2F5505A40EE7C3F355EBA15...
3	3	Apotek Kimia Farma	Apotek	Jl. Kartini No.12	0101000020E61000008D976E1283505A40D34D621058B915...
4	4	Puskesmas Rajabasa	Puskesmas	Jl. Raya Rajabasa No.5	0101000020E6100000000000000505A4085EB51B81E8515...
5	5	Pasar Bambu Kuning	Pasar	Jl. Imam Bonjol No.3	0101000020E610000046B6F3FDD4505A4083C0CAA145B61...

C. Hasil dan Kesimpulan

Praktikum ini berhasil membangun REST API WebGIS yang lengkap menggunakan FastAPI dengan koneksi ke database PostGIS. API yang dibangun memiliki 8 endpoint yang mencakup operasi CRUD lengkap dan 2 jenis query spasial yaitu pencarian berdasarkan radius (nearby) dan bounding box.

Beberapa hal penting yang dipelajari:

FastAPI memberikan kemudahan dalam membangun API karena dokumentasi otomatis, validasi input lewat Pydantic, dan performa tinggi berkat dukungan async/await.

PostGIS memungkinkan penyimpanan dan query data geografis langsung di dalam database SQL menggunakan fungsi-fungsi spasial seperti ST_DWithin, ST_AsGeoJSON, dan ST_Distance.

GeoJSON sebagai format output standar memudahkan integrasi dengan library peta di frontend seperti Leaflet untuk pertemuan berikutnya.